

Lecture: Data Visualisation

Actuarial Data Science - Open Learning Resource
Fei Huang, UNSW Sydney



Table of contents

- Exploratory Data Analysis: An Introduction
- Data Visualisation using `ggplot2`
- Declaring Data
- Aesthetic Mapping
- Facets
- Geometric Objects



Recommended Reading

- *The Art of Data Science*, Chapter 4.1–4.2
- *R for Data Science*, Chapter 1

Some coding examples in this lecture are adapted from *R for Data Science* (Wickham, Çetinkaya-Rundel, and Grolemund 2023).



Learning Objectives

- The goal of this lecture is to help you become comfortable “talking to your data”: asking simple visual questions, checking for problems, and building intuition before fitting formal models
- You should come away with a small toolkit of plots and `ggplot2` patterns that you can reuse in many different actuarial and business contexts



Learning Objectives (continued)

- Understand how to perform exploratory data analysis using the tidyverse package in R
- Explore features of the data using visualisation
- Identify common data issues, including:
 - missing values, unreliable/non-validated data, outliers, high-cardinality features
- Apply appropriate methods to address common data problems
- Assess data quality using a range of techniques
- Use R for data manipulation (e.g. filter, merge, sort, group, summarise)
- Import, export and tidy data in R
- Use R Markdown for communication and reproducible analysis



Exploratory Data Analysis: An Introduction



Exploratory Data Analysis (EDA)

- A set of procedures to produce descriptive and graphical summaries of data
- Explore the data as they are, without making strong modelling assumptions
- Examine the data to understand relationships among variables
- Detect potential problems in the dataset (e.g. outliers, missingness, miscoding)
- Check whether your question can realistically be answered using the available data
- Develop a first, rough sketch of the answer to your question, which can then be refined with more formal models



The Process of Exploratory Data Analysis

EDA is an iterative cycle. You:

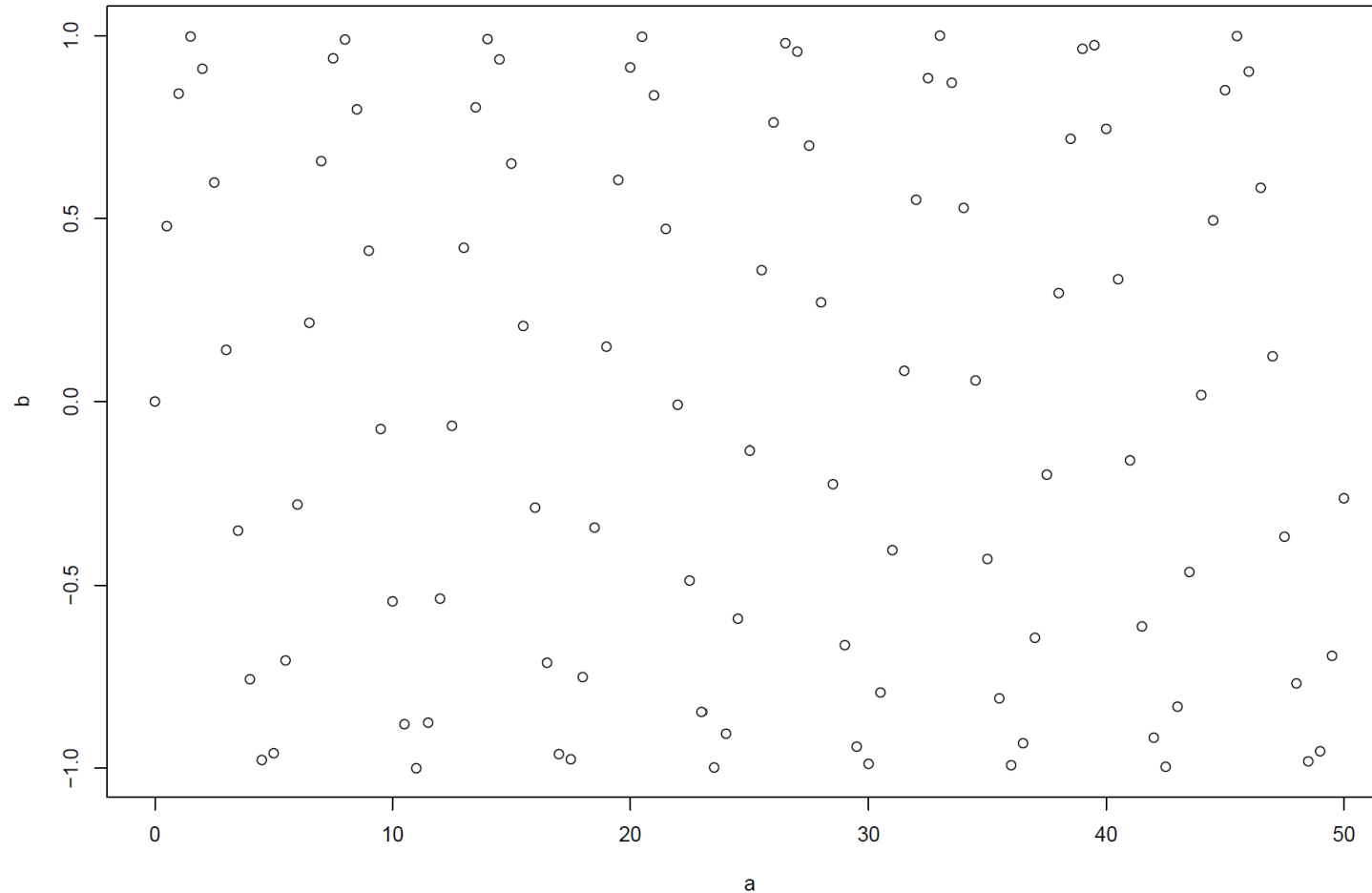
1. Formulate your question
2. Search for answers by
 - Collecting and importing data
 - Checking data quality and cleaning data
 - Manipulating and transforming data
 - Visualising data
3. Use what you learn to refine your questions and/or generate new questions

Data Visualisation is arguably the most important tool for EDA



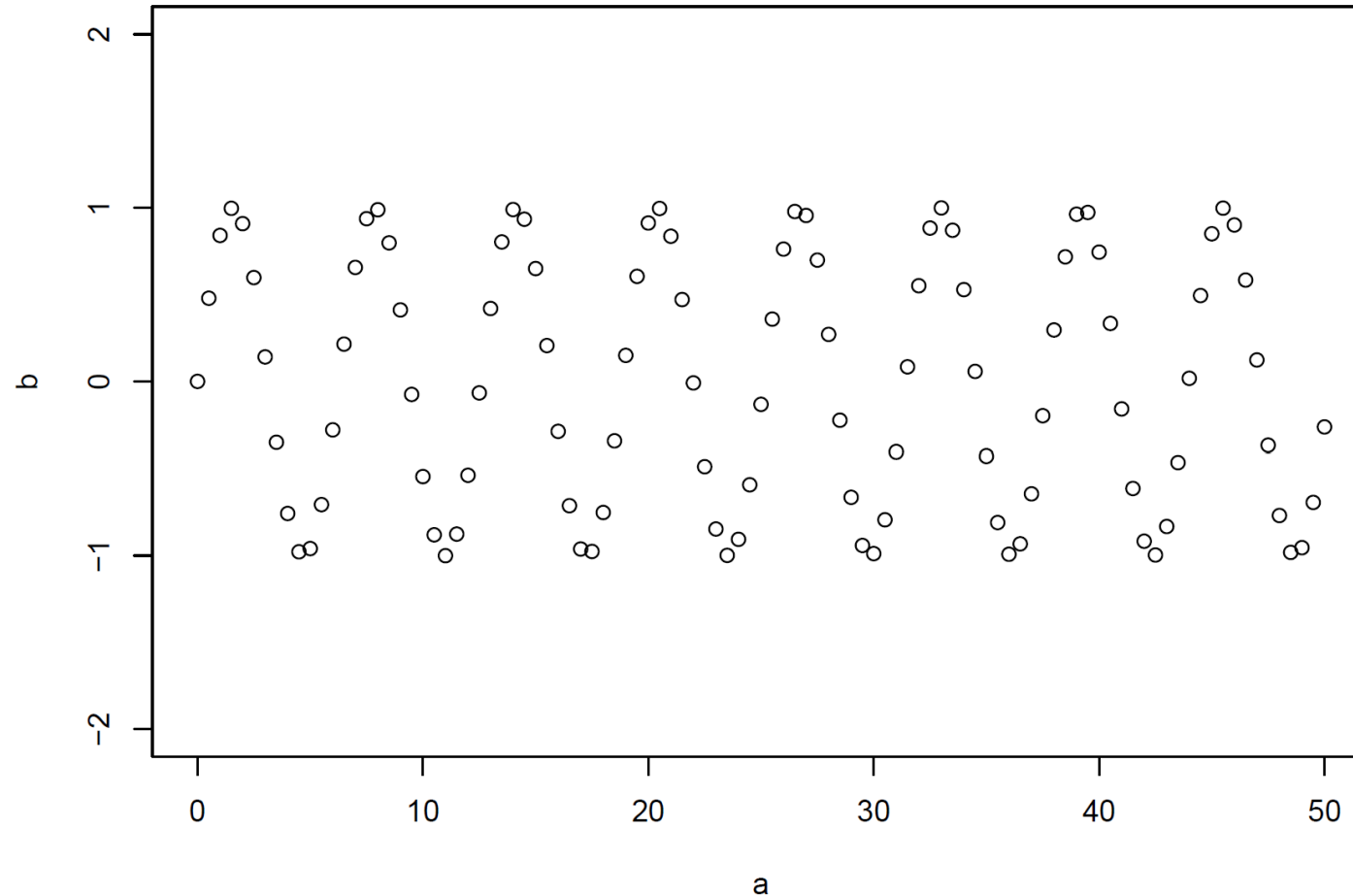
Seeing Patterns in Data

- What patterns can you see in this plot?



Seeing Patterns in Data (continued)

- What patterns can you see in this plot?



Data Visualization

“Visually attractive graphics also gather their power from content and interpretations beyond the immediate display of some numbers. The best graphics are about the useful and important, about life and death, about the universe. Beautiful graphics do not traffic with the trivial.”

— Edward Tufte, *The Visual Display of Quantitative Information*



Data Visualisation using ggplot2



Explore Features of Data using Visualisation

- A *statistical graphic* maps variables from
 1. a *dataset* to
 2. *aesthetic properties* of
 3. *geometric objects*.
- `ggplot2` is part of the `tidyverse`
- Use `ggplot2` to visualise your data
- See also: [A ggplot2 grammar guide](#)



Loading ggplot2

```
1 #install.packages("tidyverse")  
2 library(tidyverse)
```



- Reload the package every time you start a new session
- Use `package::function()` to call functions explicitly, e.g. `ggplot2::ggplot()`



A First Visualisation Question: The mpg Dataset

- Question: Do cars with larger engines use more fuel than cars with small engines?
- Data: the `mpg` data frame in `ggplot2` (`ggplot2::mpg`)
 - `mpg` contains observations collected by the US Environmental Protection Agency on 38 popular models of cars from 1999 to 2008
 - The data frame has 234 rows and 11 variables



A First Visualisation Question: The mpg Dataset (continued)

- Variables:
 - **manufacturer**: manufacturer name
 - **model**: model name
 - **displ**: engine displacement, in litres
 - **year**: year of manufacture
 - **cyl**: number of cylinders
 - **trans**: type of transmission
 - **drv**: type of drive train, where **f** = front-wheel drive, **r** = rear wheel drive, **4** = four-wheel drive
 - **cty**: city miles per gallon
 - **hwy**: highway miles per gallon
 - **fl**: fuel type
 - **class**: vehicle class



Inspecting the mpg Dataset

1 mpg 

manufacturer <chr>	model <chr>	displ <dbl>	year <int>	cyl <int>	
audi	a4	1.8	1999	4	
audi	a4	1.8	1999	4	
audi	a4	2.0	2008	4	
audi	a4	2.0	2008	4	
audi	a4	2.8	1999	6	
audi	a4	2.8	1999	6	
audi	a4	3.1	2008	6	
audi	a4 quattro	1.8	1999	4	
audi	a4 quattro	1.8	1999	4	
audi	a4 quattro	2.0	2008	4	

1-10 of 234 rows | 1-5 of 11 columns

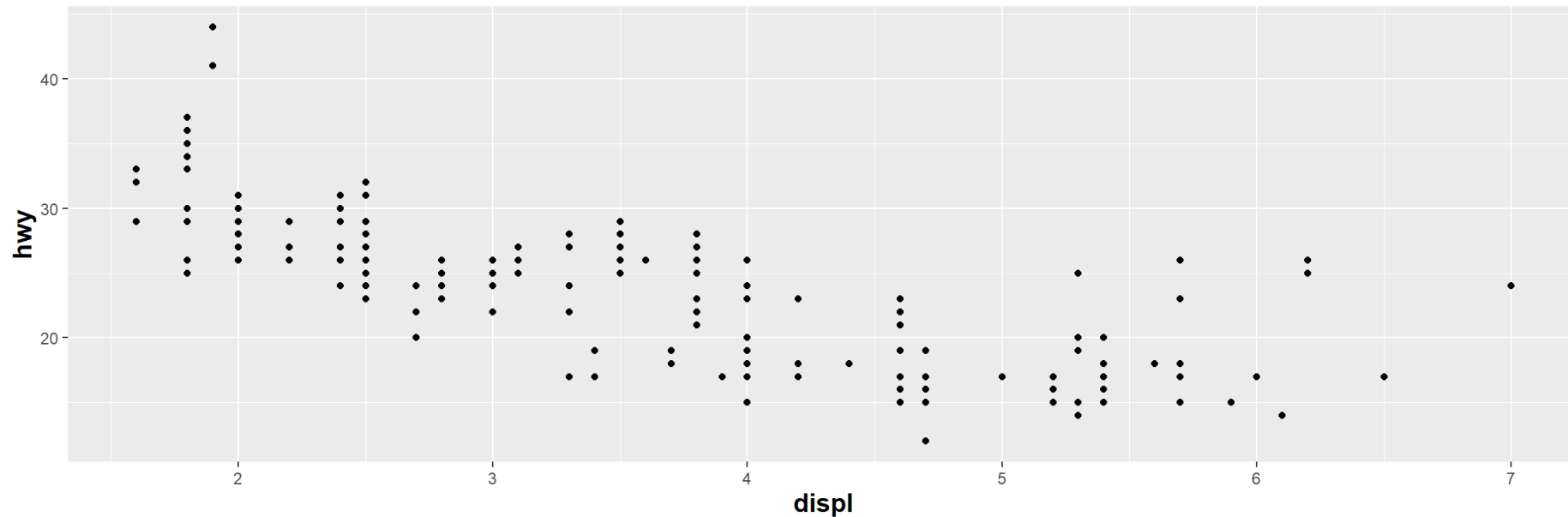
Previous 1 2 3 4 5 6 ... 24 Next

See also: More aesthetic mappings



Creating a Plot

```
1 ggplot(data = mpg) + # the dataset
2   aes(x = displ) + # the x position
3   aes(y = hwy) +
4   # the y position
5   geom_point() + # the point geometric shape
6   # Adjust axis titles' front size
7   theme(axis.title=element_text(size=14,face="bold"))
```



See also: [A complete ggplot “sentence”](#); [Local mpg plot](#)

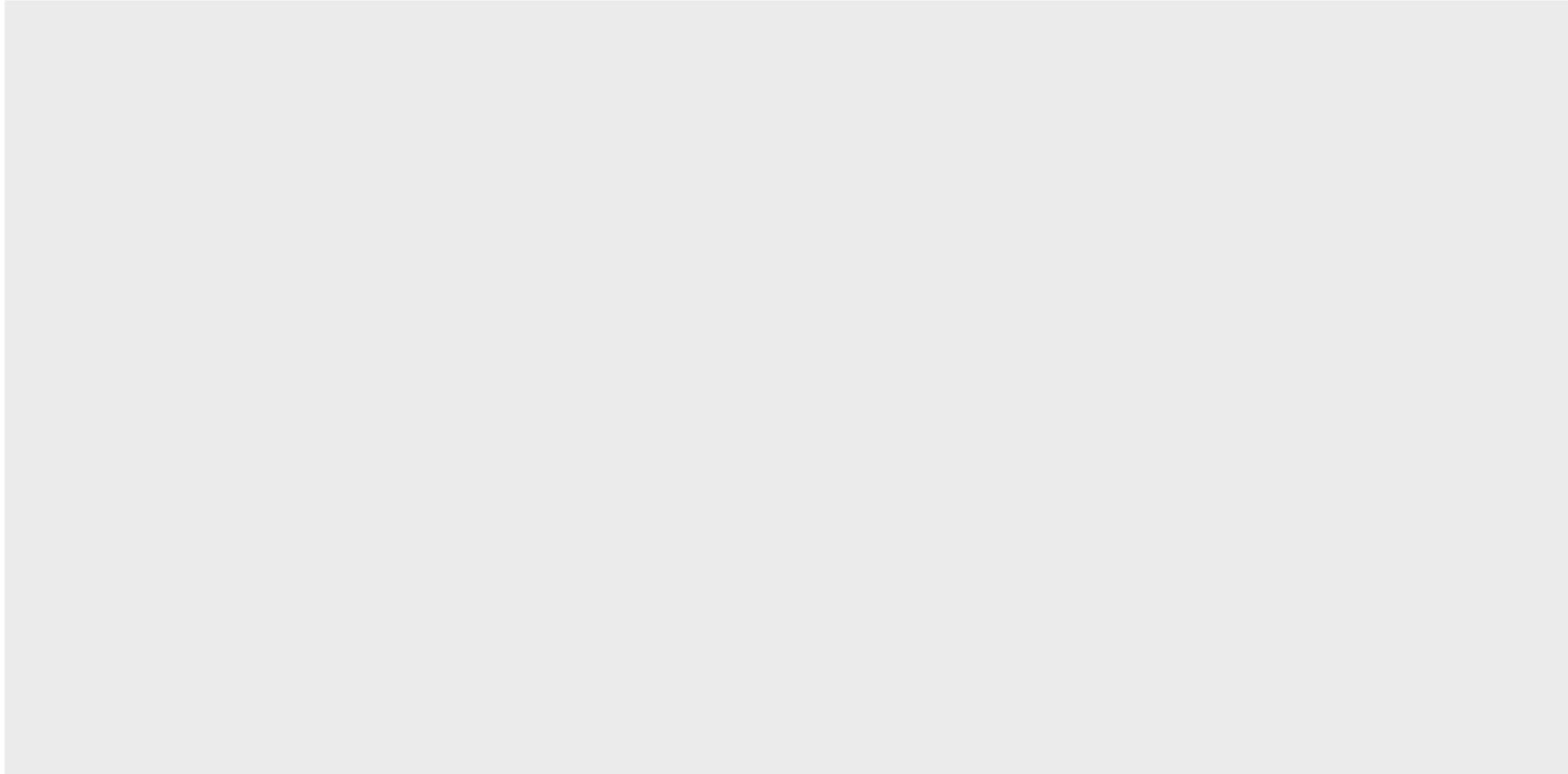


Declaring Data



Method 1: Declare Data Inside ggplot()

```
1 ggplot(data = mpg)
```



Method 2: Use the Pipe Operator

- Pipe data into `ggplot()` using the pipe operator `%>%`

```
1 mpg %>% # data piped into  
2   ggplot() # initiating plot
```



Exercises: Declaring Data

1. Run `ggplot(data = mpg)`. What do you see?
2. How many rows and columns are in `mpg`?
3. What does the `drv` variable describe? Read the help for `?mpg` to find out.
4. Create a scatterplot of `hwy` vs `cyl`.
5. What happens if you make a scatterplot of `class` vs `drv`? Why is the plot not useful?



Aesthetic Mapping

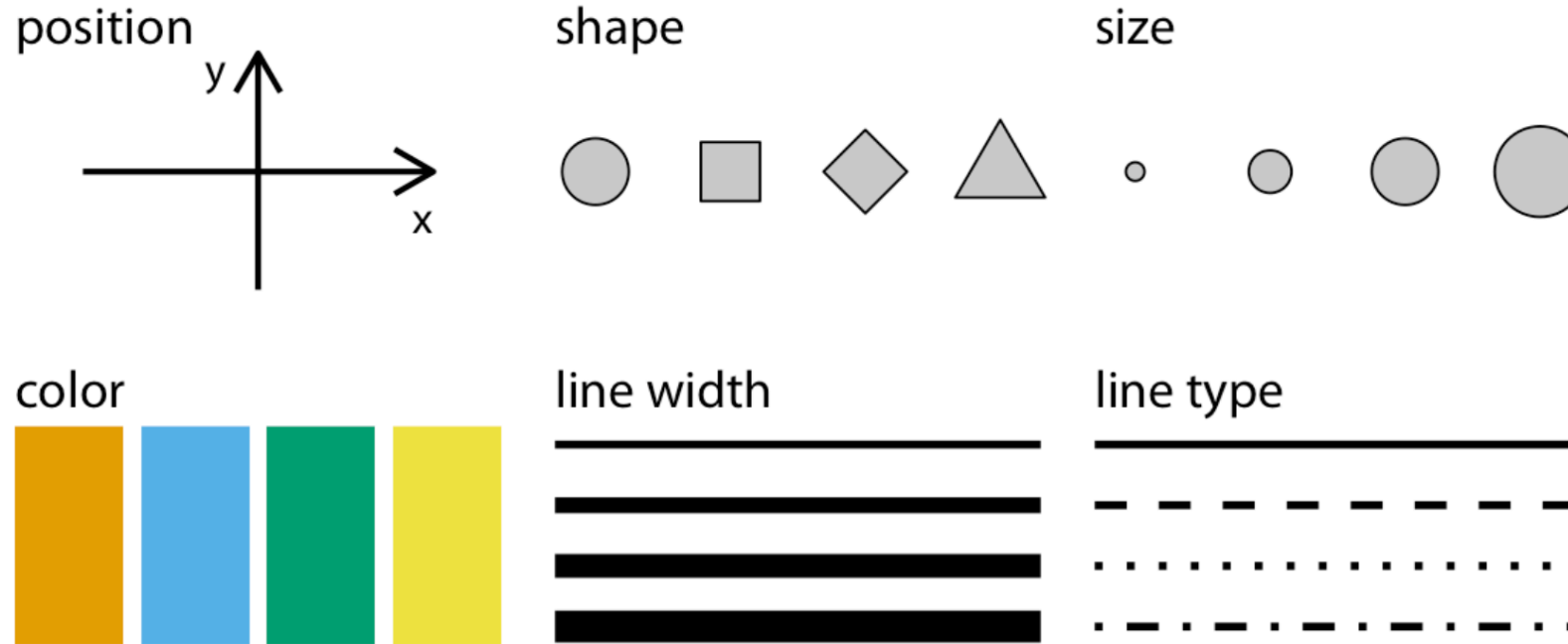


What is an Aesthetic Mapping?

- An aesthetic is a visual property of the objects in your plot.
- Aesthetics include properties such as *position*, *size*, the *shape*, and *colour*.
- Mapping refers to linking variables in the data to visual properties (aesthetics) in the plot.



Common Aesthetic Mappings



A main pool of aesthetics (Source: Wilke (2019), *Fundamentals of Data Visualization*)

`aes()` means 'Ask'

- `aes()`: What variables are we asking the aesthetics (colour, position, shape, etc.) to represent?
- `aes(colour = gender)`: "Please represent the variable `gender` using different colours."



Mapping Multiple Aesthetics

See also: The mpg dataset

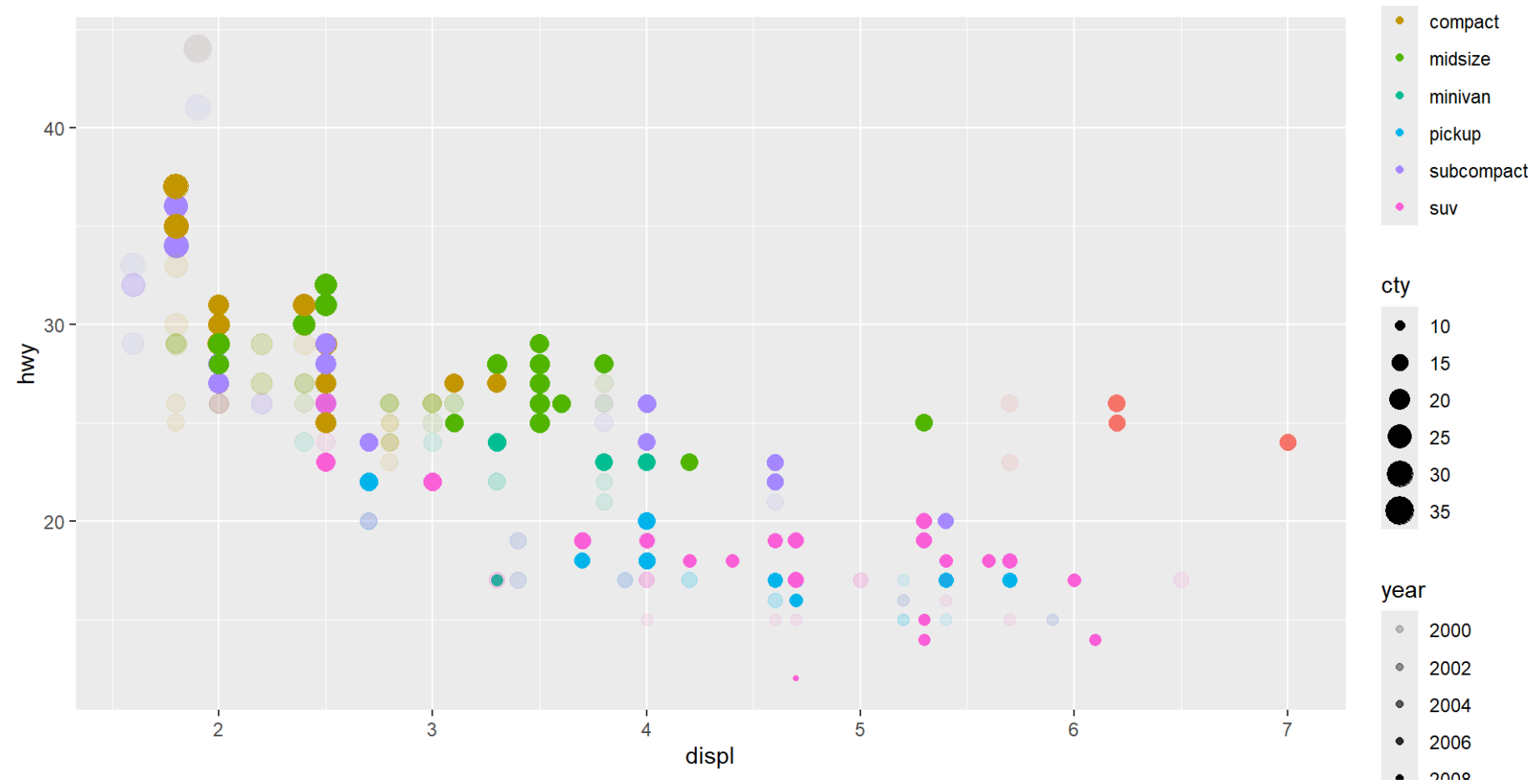
```
1 mpg_plot= ggplot(data = mpg) + # the dataset
2   aes(x = displ) + # the x position
3   aes(y = hwy) + # the y position
4   geom_point() +
5   #the point geometric shape, the above aes are required and
6   #the below are optional
7   #theme(axis.title=element_text(size=14,face="bold"))+
8   aes(colour = class) + # colour for type of car
9   #aes(shape = class) +
10  #ggplot2 will only use six shapes at a time. By default,
11  #additional groups will go unplotted when using 'shape'.
12  aes(size = cty) + # Size for city miles per gallon
13  aes(alpha = year) # transparency for year of manufacture
```

See also: A complete ggplot “sentence”



Example: Multiple Aesthetic Mappings

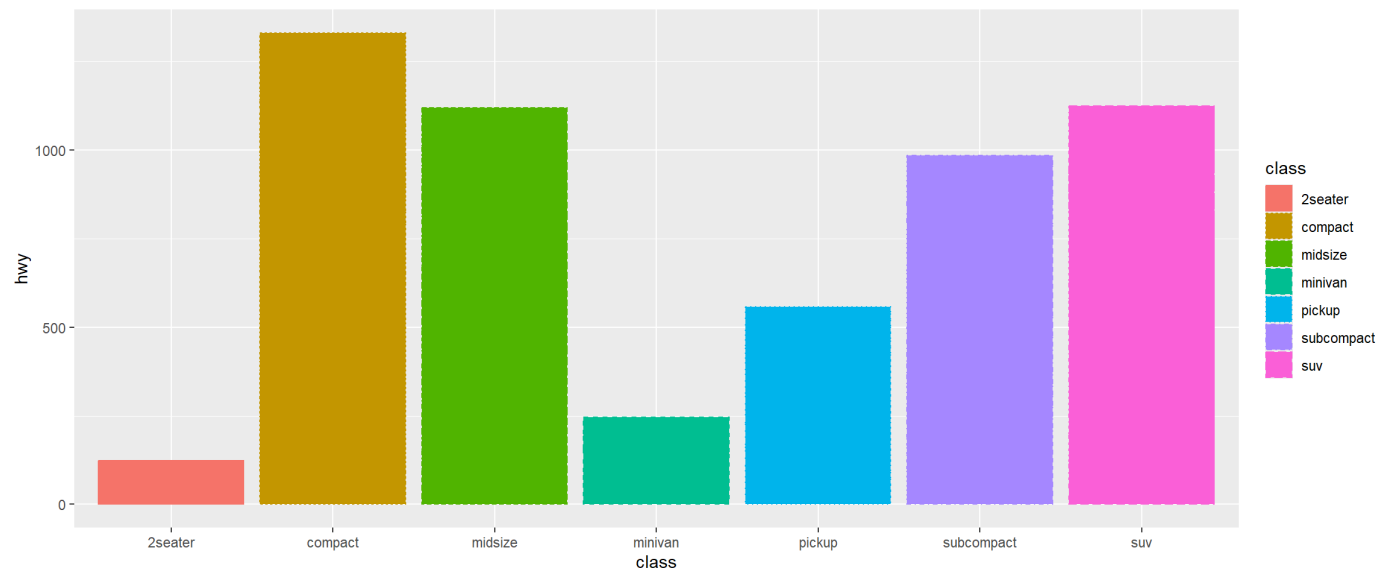
```
1 print(mpg_plot)
```



Aesthetic Mappings for Different Geoms

Different geometric objects use different aesthetic properties. For example, column plots often use `fill` to distinguish groups.

```
1 mpg %>% # data piped into
2   ggplot() + # initiating plot
3   aes(x = class) + #categorical variable
4   aes(y = hwy) +
5   geom_col() + #Use `geom_col` to create a column geometry
6   aes(colour = class) +
7   aes(fill = class) + # new aes 'fill'
8   aes(linetype = class) #new aes 'linetype'
```



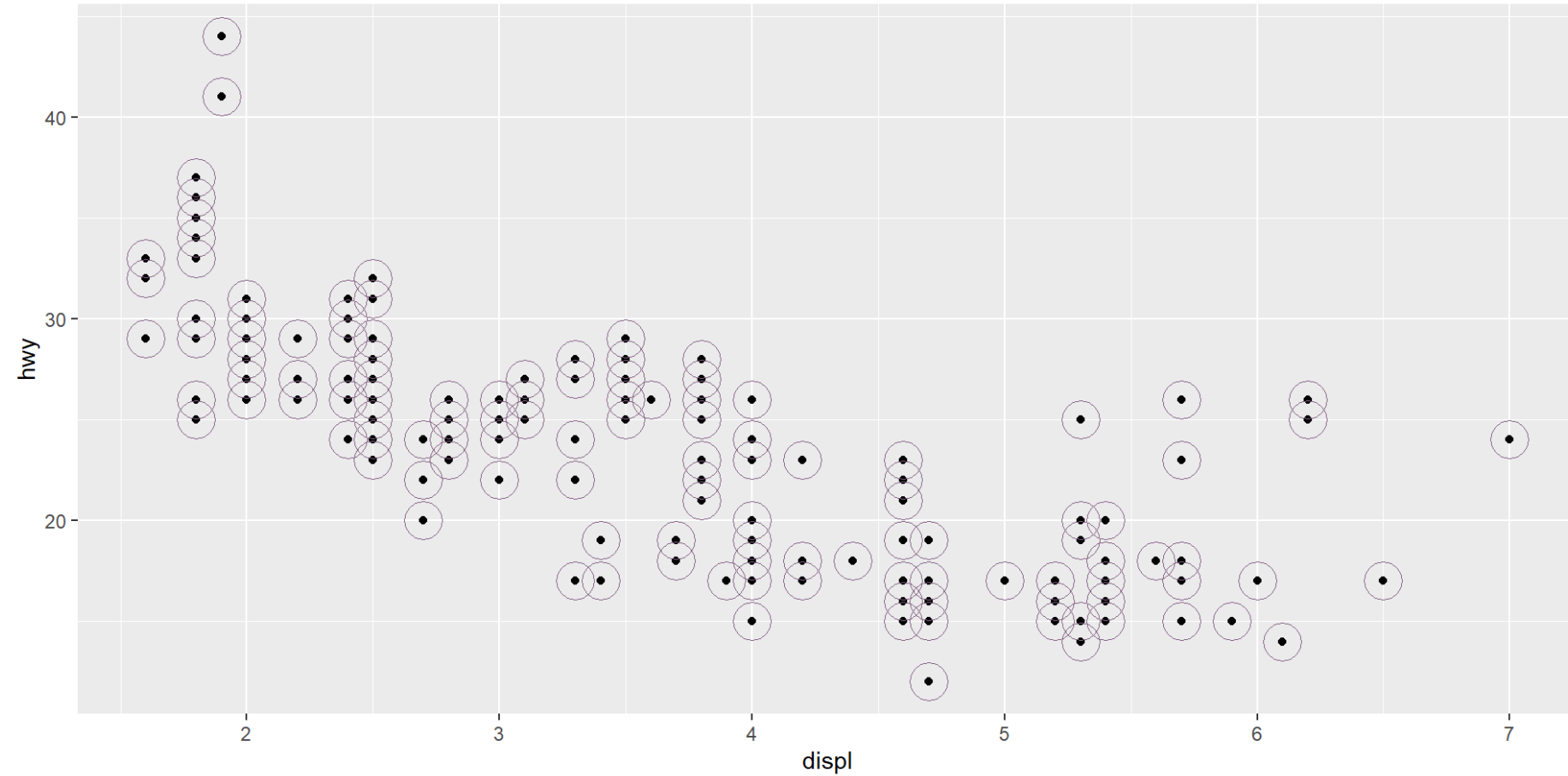
Unmapped Aesthetics

```
1 # Create scatter plot with base layer
2 mpg_plot <- ggplot(data = mpg) +
3   aes(x = displ, y = hwy) +
4   geom_point() +
5   # Add another point layer with fixed (unmapped) aesthetics
6   # These aesthetics don't represent variables - they're constant values
7   geom_point(
8     colour = "plum4", # Fixed colour for all points in this layer
9     size = 8,         # Fixed size for all points
10    shape = 21         # Fixed shape (filled circle)
11  )
```



Plot: Unmapped Aesthetics

```
1 print(mpg_plot)
```

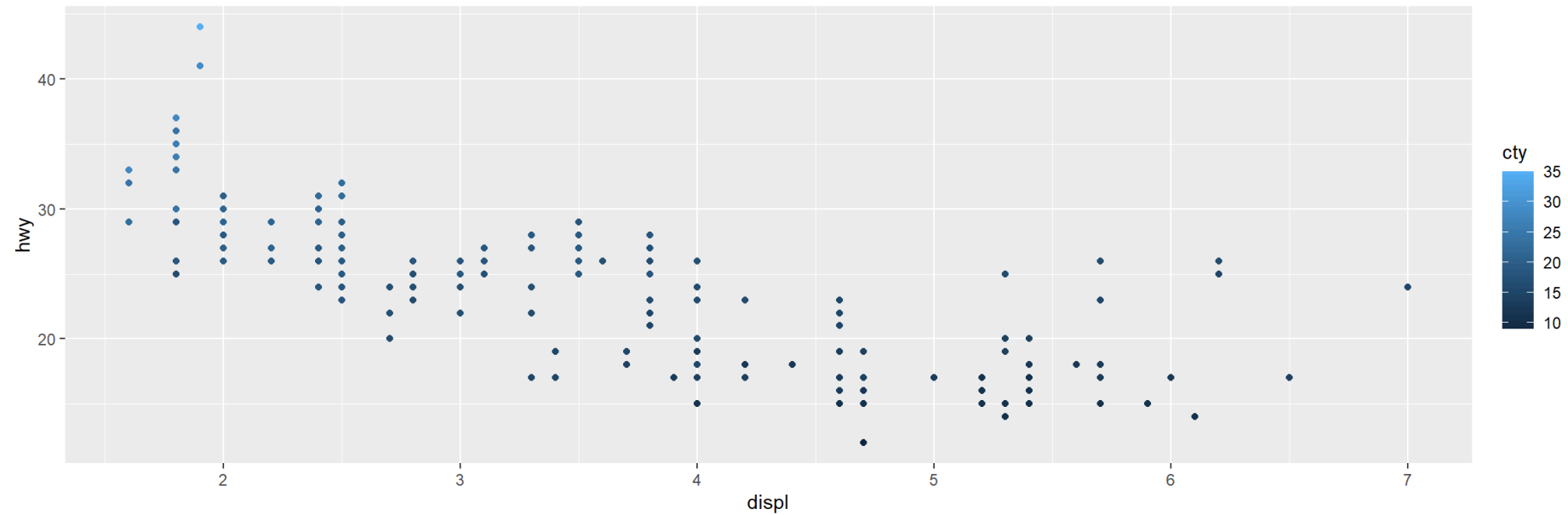


Exercises: Aesthetic Mappings

1. Look at the `help` for `geom_text` (`?geom_text`). What are the required aesthetics?
2. Which variables in `mpg` are categorical, and which are continuous? (Hint: use `?mpg` to read the dataset documentation). How can you identify this when you run `mpg`?
3. Map a continuous variable to colour, size, and shape. How do these aesthetics behave differently for categorical versus continuous variables?




```
1 ggplot(data = mpg) + # the dataset
2   aes(x = displ) + # the x position
3   aes(y = hwy) +
4   aes(colour = cty) + #colour, size and shape
5   # the y position
6   geom_point() # the point geometric shape
```



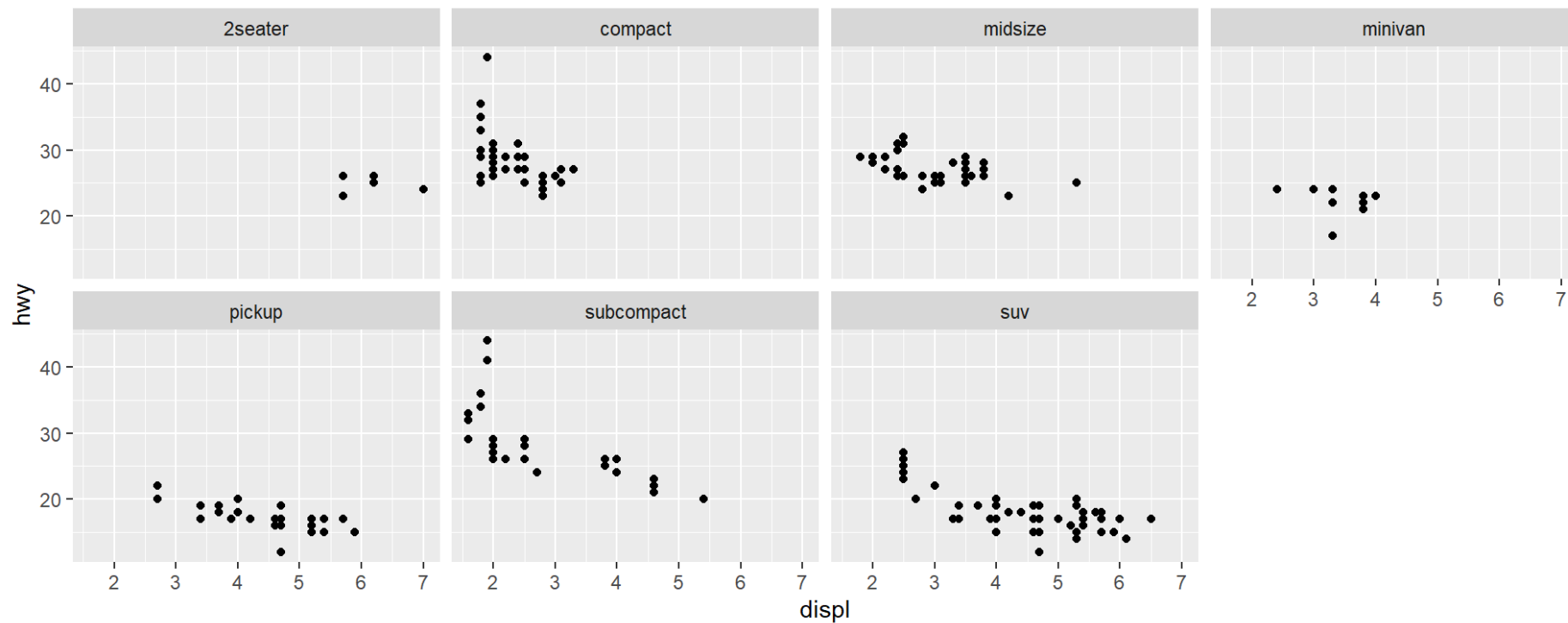
Facets



Using facet_wrap()

- Facet your plot by a single variable

```
1 ggplot(data = mpg) +  
2   geom_point(mapping = aes(x = displ, y = hwy)) +  
3   # ~ followed by a discrete variable  
4   facet_wrap(~ class, nrow = 2)
```

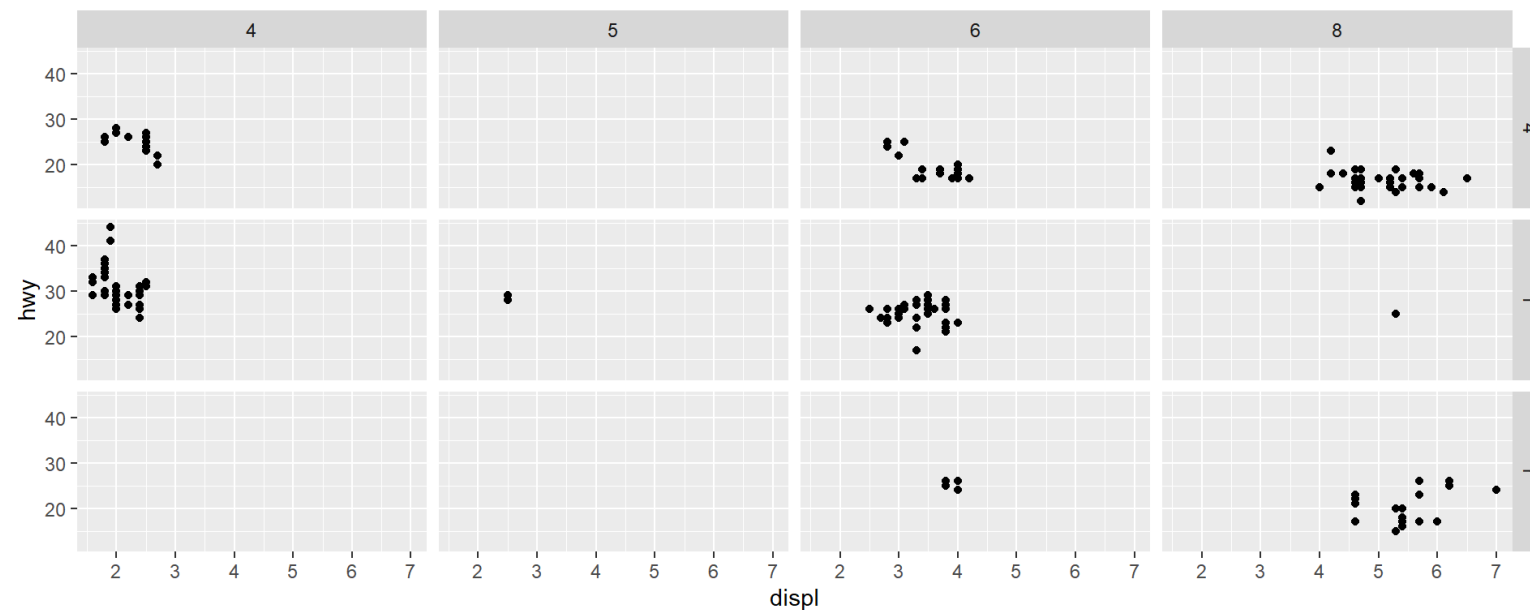


Using facet_grid()

- Facet your plot by the combination of two variables

```
1 ggplot(data = mpg) +
2   geom_point(mapping = aes(x = displ, y = hwy)) +
3   # two variable names separated by a ~
4   facet_grid(drv ~ cyl)
```

```
1 #facet_grid(. ~ cyl) #not facet in the rows
2 #facet_grid(drv~.) #not facet in the rows
```



Geometric Objects



A Complete “Sentence” in `ggplot`

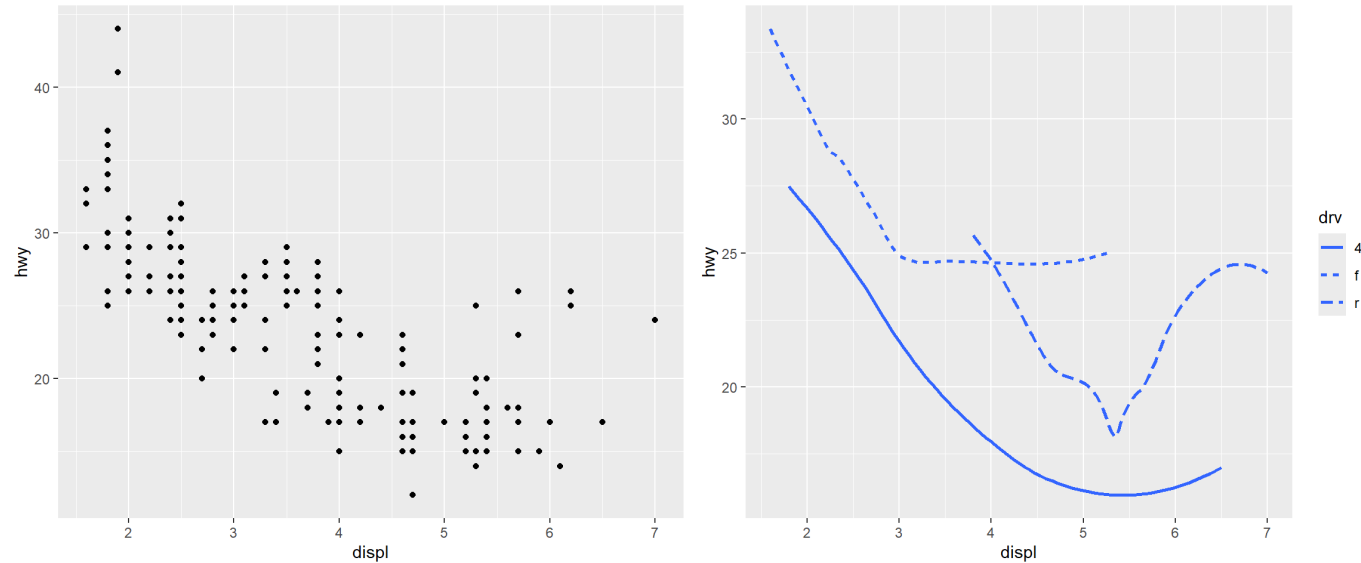
- `data` + `aes` + `geom`
- Plots:
 - `mpg` plot
 - more aesthetic mappings
- Nouns: geometric objects
 - `geom_point()`
 - `geom_col()`
 - `geom_line()`
 - `geom_text()`
 - `geom_segment()`
 - `geom_smooth()`
 - `geom_bar()`
 - etc.
- The conditional mood: `geom` specific data and aesthetic mapping



Different Geoms

Compare with **mpg plot**

```
1 library(grid)
2 library(gridExtra)
3 # Create scatter plot with points
4 scatter_plot <- ggplot(data = mpg) +
5   geom_point(mapping = aes(x = displ, y = hwy))
6 # Create smooth line plot with different linetypes by drive type
7 smooth_plot <- ggplot(data = mpg) +
8   geom_smooth(mapping = aes(x = displ, y = hwy, linetype = drv), se = FALSE)
9 # Arrange both plots side by side
10 grid.arrange(scatter_plot, smooth_plot, ncol = 2)
```

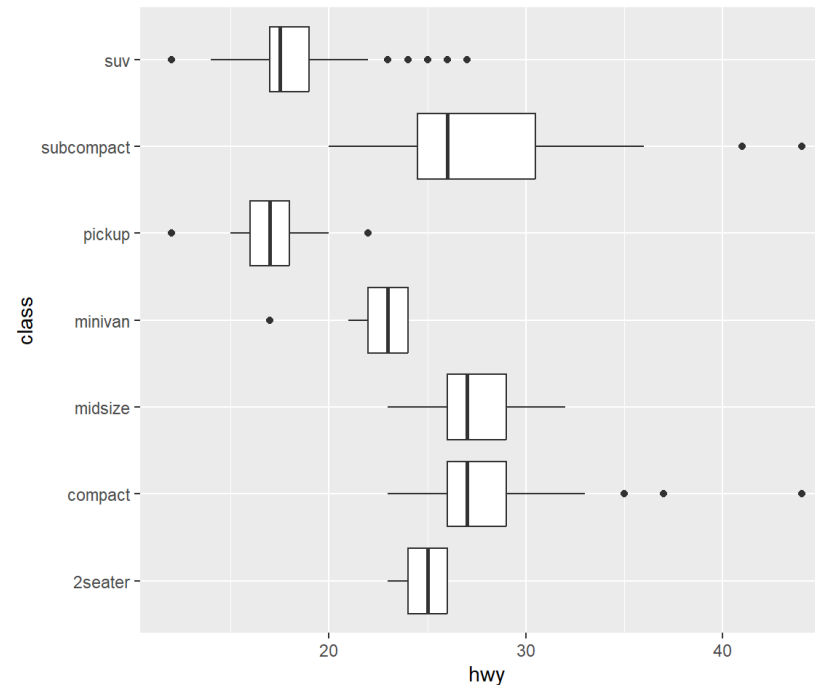
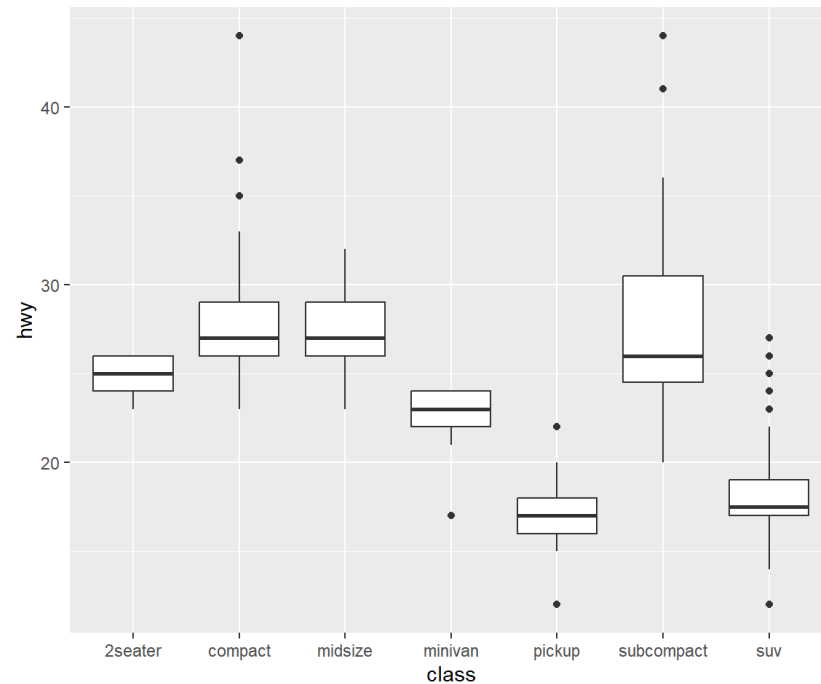


Example: Boxplots

```

1 # Boxplot with default orientation (vertical)
2 boxplot_vertical <- ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
3   geom_boxplot()
4 # Boxplot with flipped axes (horizontal)
5 boxplot_horizontal <- ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
6   geom_boxplot() +
7   coord_flip() # switches the x and y axes
8 # Arrange both plots side by side for comparison
9 grid.arrange(boxplot_vertical, boxplot_horizontal, ncol = 2)

```



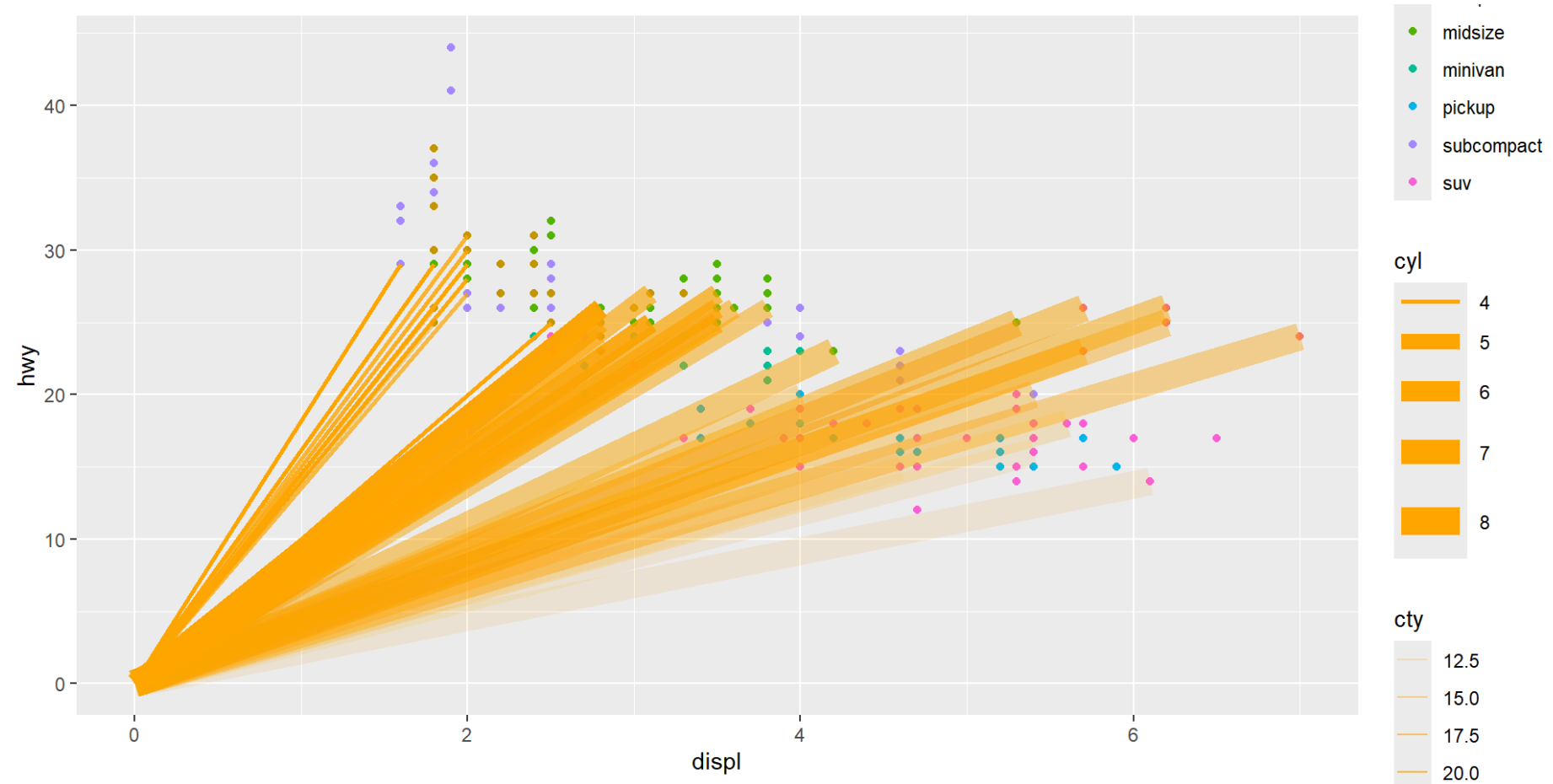
Local Data and Aesthetic Mappings

```
1 # Create base scatter plot with global aesthetics
2 mpg_plot <- ggplot(data = mpg) +
3   aes(x = displ, y = hwy) +
4   geom_point() +
5   aes(colour = class) +
6   # xend and yend are required for geom_segment
7   # Set endpoints to origin (0, 0) for all segments
8   aes(xend = 0, yend = 0) +
9   # Add segments layer with local data and aesthetics
10  # geom_segment() draws a straight line between points (x, y) and (xend, yend)
11  geom_segment(
12    # Use local data: only flights with premium fuel (fl == "p")
13    data = subset(mpg, fl == "p"),
14    # Local aesthetics: size mapped to cylinders, alpha to city mpg
15    aes(size = cyl, alpha = cty),
16    colour = "orange" # Fixed colour for all segments
17  )
```



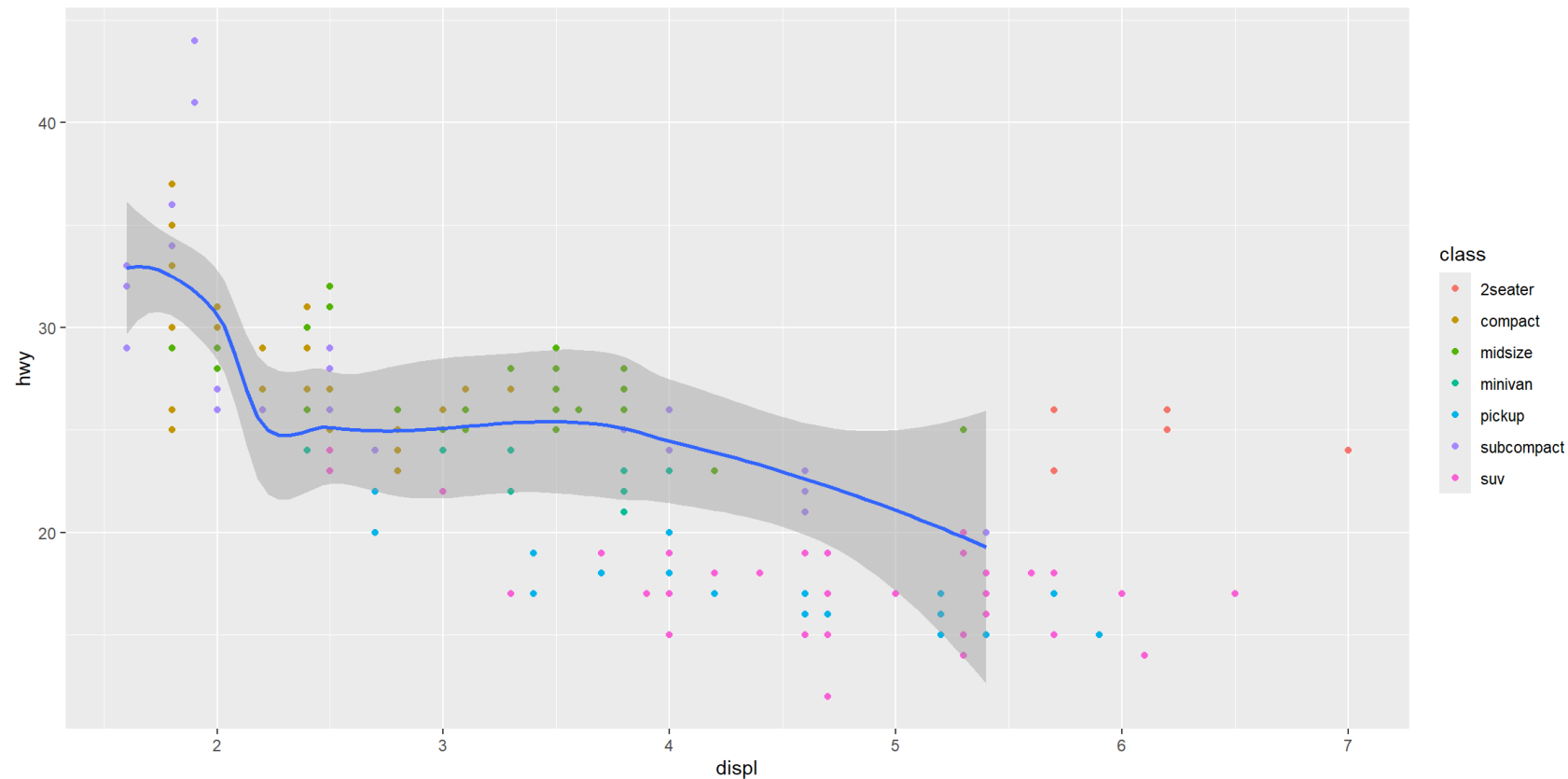
Result: Local Data and Aesthetic Mappings

```
1 print(mpg_plot)
```



Example: Local Aesthetic Mappings

```
1 ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) + #global aes
2   geom_point(mapping = aes(colour = class)) + #local aes
3   #local data and aes
4   geom_smooth(data = filter(mpg, class == "subcompact"),
5               se = TRUE) #se: standard error
```



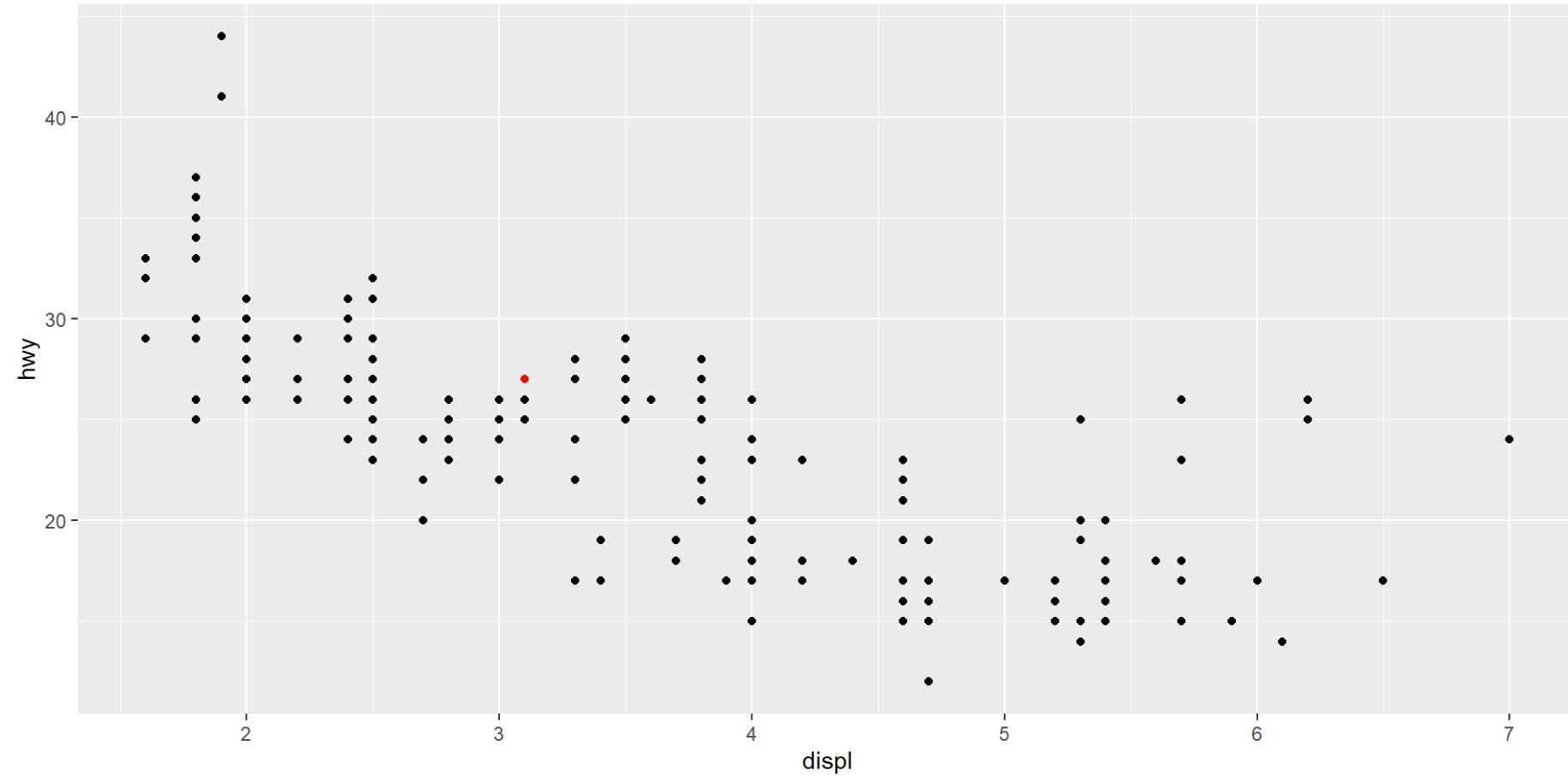
Annotation: Highlighting a Point

```
1 # Create base scatter plot
2 mpg_plot <- ggplot(data = mpg) +
3   aes(x = displ, y = hwy) +
4   geom_point() +
5   # Add annotation: highlight a specific point
6   annotate(geom = "point",
7           x = 3.1,      # x-coordinate of highlighted point
8           y = 27,      # y-coordinate of highlighted point
9           colour = "red")
```



Result: Highlighting a Point

```
1 print(mpg_plot)
```



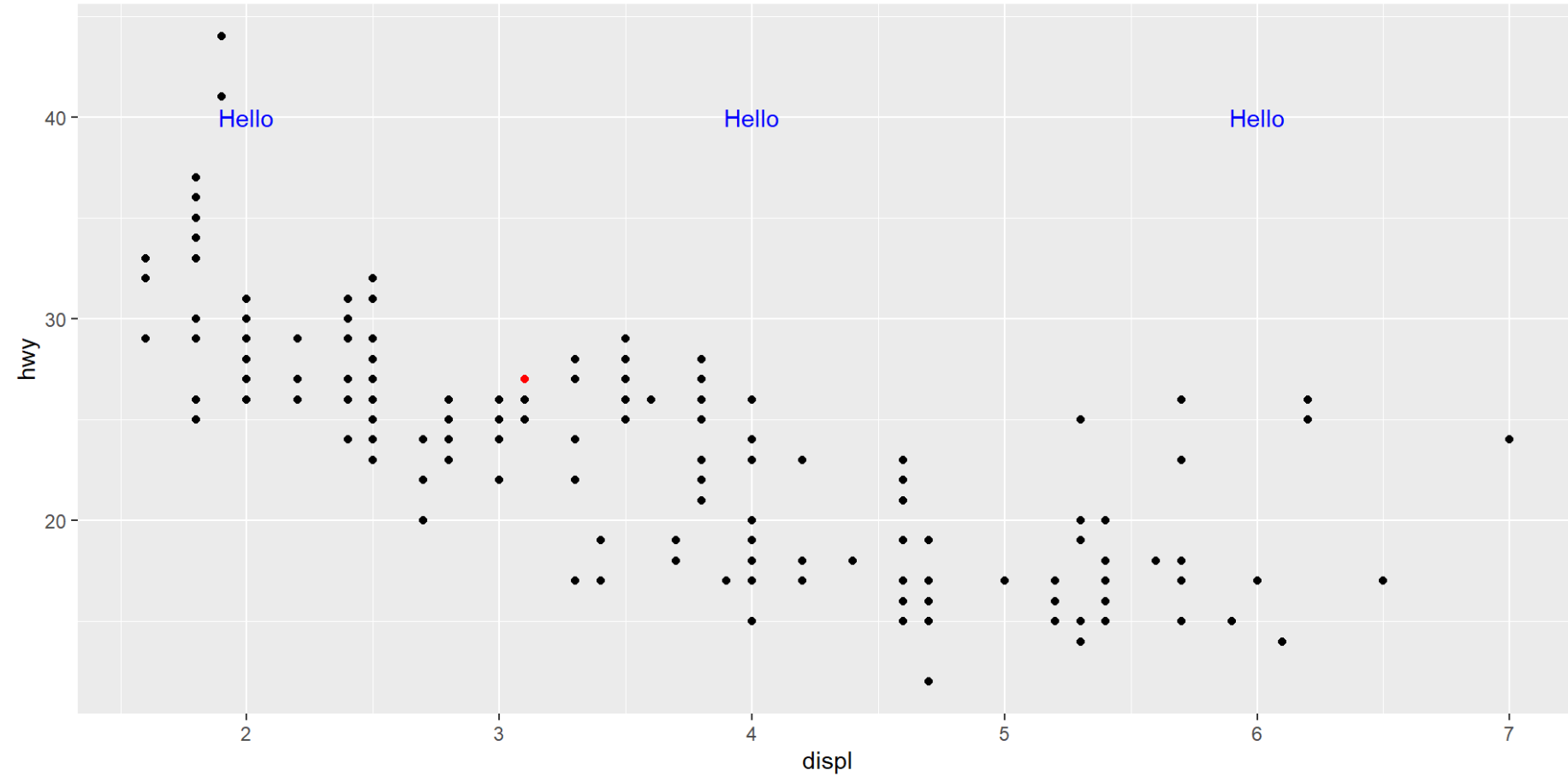
Annotation: Adding Text

```
1 # Create base scatter plot
2 mpg_plot <- ggplot(data = mpg) +
3   aes(x = displ, y = hwy) +
4   geom_point() +
5   # Add highlighted point annotation
6   annotate(geom = "point",
7           x = 3.1, y = 27,
8           colour = "red") +
9   # Add text annotations at multiple locations
10  annotate(geom = "text",
11          x = c(2, 4, 6), # Multiple x-coordinates
12          y = 40,         # Single y-coordinate (text appears at same height)
13          label = "Hello",
14          colour = "blue")
```



Result: Adding Text

```
1 print(mpg_plot)
```



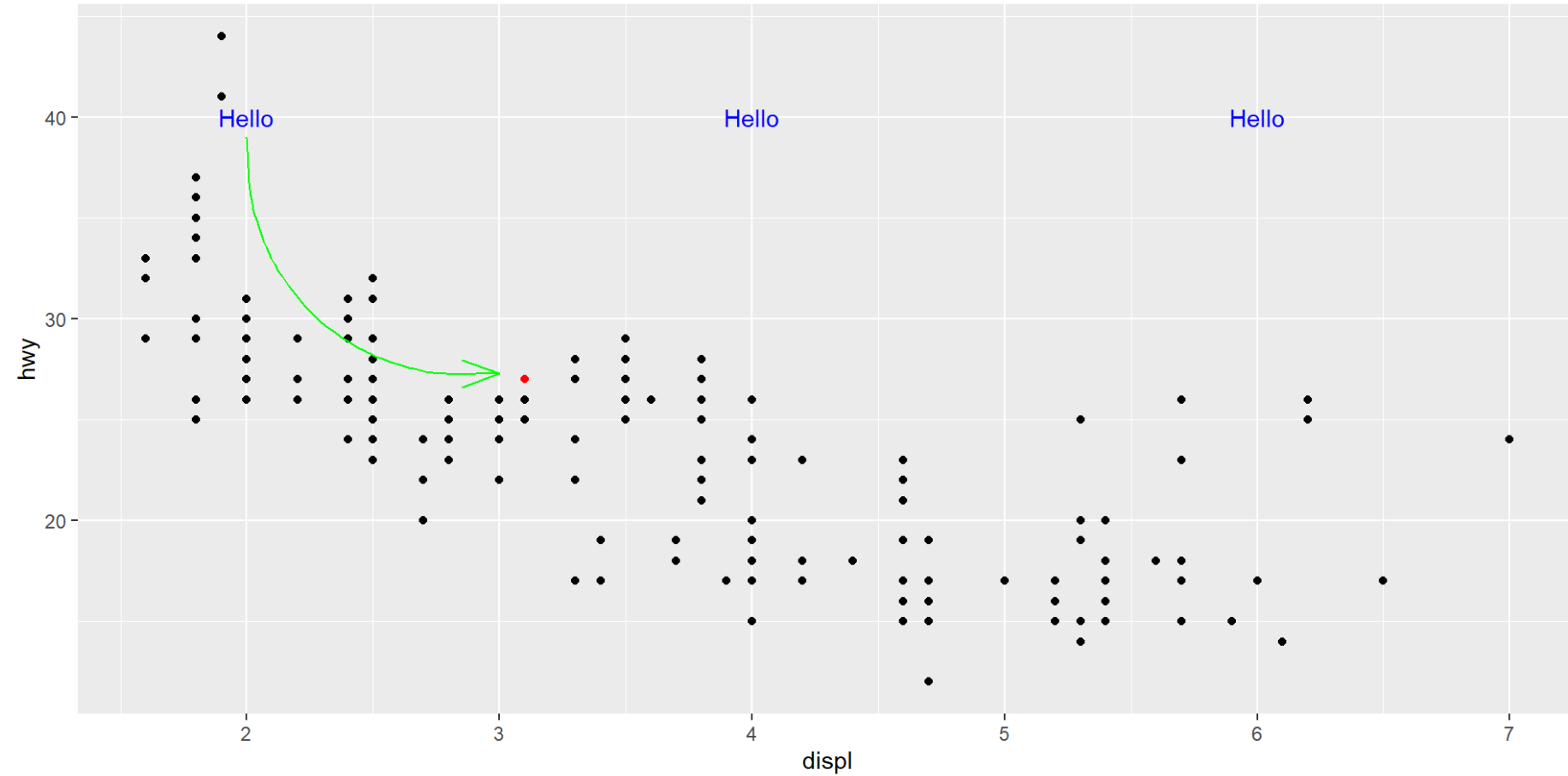
Annotation: Adding a Curved Arrow

```
1 # Create base scatter plot
2 mpg_plot <- ggplot(data = mpg) +
3   aes(x = displ, y = hwy) +
4   geom_point() +
5   # Highlight a specific point
6   annotate(geom = "point",
7           x = 3.1, y = 27,
8           colour = "red") +
9   # Add text labels
10  annotate(geom = "text",
11          x = c(2, 4, 6), y = 40,
12          label = "Hello",
13          colour = "blue") +
14  # Add curved arrow connecting text to point
15  annotate(geom = "curve",
16          x = 2, y = 39,          # Starting point of arrow
17          xend = 3, yend = 27.3, # Ending point of arrow
18          colour = "green",
19          arrow = arrow(angle = 20)) # Arrow head angle
```



Result: Adding a Curved Arrow

```
1 print(mpg_plot)
```



Annotation: Adding Reference Lines

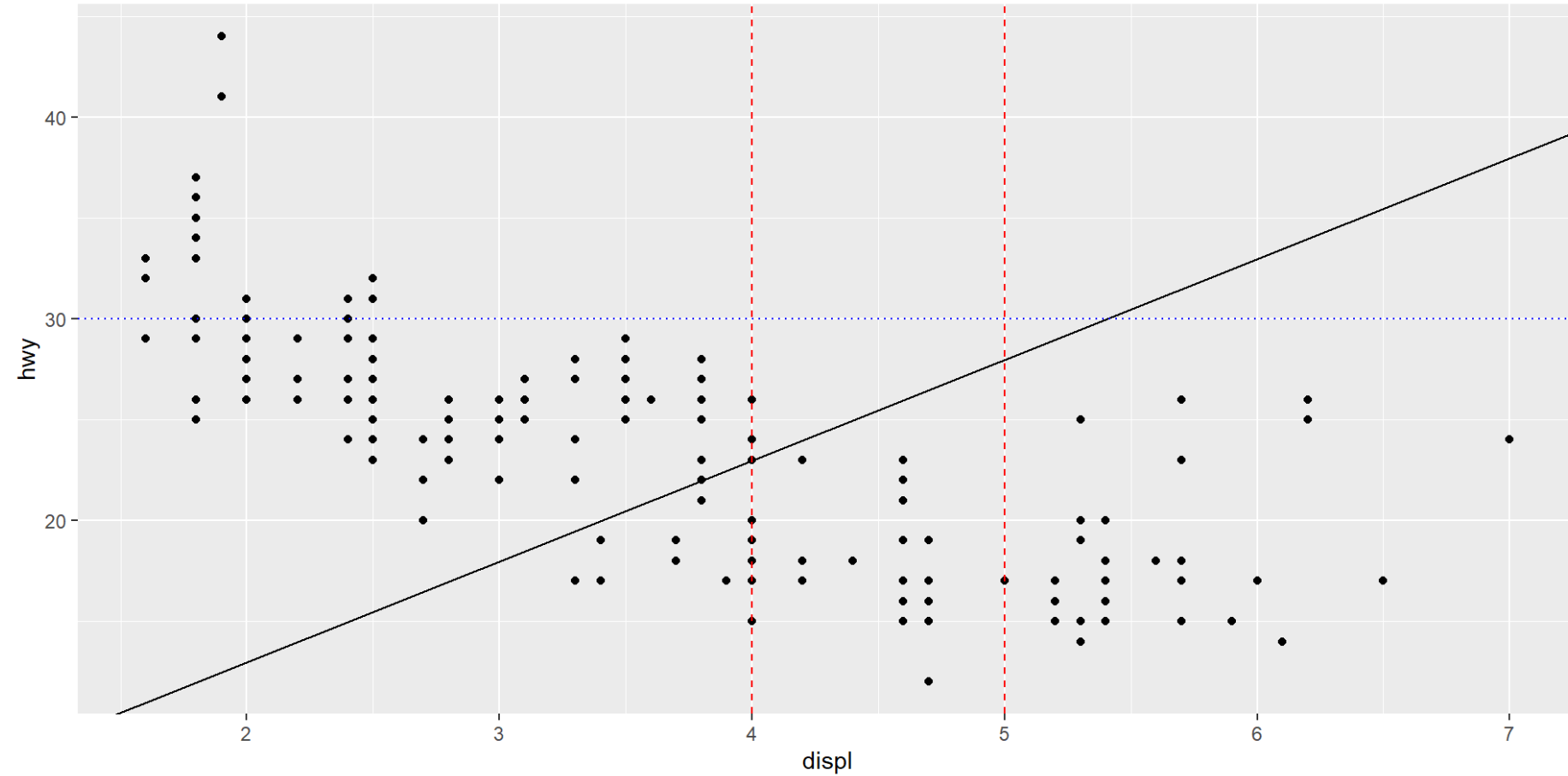
- use `geom_abline`, `geom_hline`, and `geom_vline`

```
1 # Create base scatter plot
2 mpg_plot <- ggplot(data = mpg) +
3   aes(x = displ, y = hwy) +
4   geom_point() +
5   # Add reference lines
6   geom_abline(slope = 5, intercept = 3) + # Diagonal line: y = 5x + 3
7   geom_hline(yintercept = 30,           # Horizontal line at y = 30
8             linetype = "dotted",
9             colour = "blue") +
10  geom_vline(xintercept = c(4, 5),      # Vertical lines at x = 4 and x = 5
11            linetype = "dashed",
12            colour = "red")
```



Result: Adding Reference Lines

```
1 print(mpg_plot)
```



Exercises: Aesthetics and Annotations

1. What has gone wrong with this code? Why are the points not blue?

```
1 ggplot(data = mpg) +  
2   geom_point(mapping = aes(x = displ,  
3                             y = hwy, colour = "blue"))
```

2. What happens if you map an aesthetic to something other than a variable name, e.g. `aes(colour = displ < 5)`? Note: you will also need to specify `x` and `y`.



Interactive Data Visualisation (Optional Extension)

- R package: *Shiny*
- Can host standalone apps on a webpage
 - Example: *Life Expectancy* using data from the
- Can embed apps in R Markdown documents or build dashboards



References

- Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. 2023. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. 2nd ed. O'Reilly Media.
- Wilke, Claus O. 2019. *Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures*. O'Reilly Media.

